Reporte

Nombre: Equipo 6 Luis David Cigala Montes Jared Alejandro Garcia Alcocer Gabriel Sánchez Rivera	**Matrícula**: 2120993 2119041 2121131
Nombre del curso: Componentes de lógica de negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 3	**Actividad**: Actividad 3 service layer
Fecha: 10/15/26	
Bibliografia (Formato APA):Archivo de la actividad 3 brindada por el ingeniero	

Service Layer

Servicio	Método o caso de uso	Entrada
OrdenServicioAppService	RegistrarOrdenServicio	clienteId, datosVehiculo, checklistRecepcion, canal, responsableId
OrdenServicioAppService	CerrarOrdenServicio	ordenId, pagoCompletoConfirmado, firmaConformidadCliente
CotizacionAppService	RegistrarAutorizacionCotizacion	cotizacionId, tokenUnico, respuestaCliente
CotizacionAppService	GestionarTrabajoAdicional	ordenId, costoAdicionalDetectado
ResguardoAppService	ProcesarCargoAlmacenaje	ordenId, diasTranscurridos, llamadasRealizadas

Colaboradores	Salida	Reglas que delega
ClienteRepository, VehiculoRepository, OrdenServicioRepository, ControladorDeBahias	ordenId	1. Valida la existencia del cliente y vehículo. 2. Delega al Domain Service ControladorDeBahias verificar disponibilidad en taller (máx. 12 bahías, 8 cajones) (US-11). 3. Crea la entidad OrdenDeServicio vía método de fábrica en el dominio. 4. Guarda con OrdenServicioRepository.guardar(). 5. Confirma la transacción con UnitOfWork.commit().
OrdenServicioRepository, BitacoraAuditoriaRepository, NotificacionService	estadoCierre	. Busca OrdenDeServicio por ID. 2. Llama a orden.verificarCondicionTripleCierre() donde el dominio valida que el vehículo esté reparado, pagado y firmado (US-02).

		3. Registra el evento en la bitácora inalterable (US-19). 4. Guarda la orden actualizada. 5. Dispara notificación y ejecuta UnitOfWork.commit().
CotizacionRepository, OrdenServicioRepository, ValidadorDeAnticipos	estatusAutorizacionId	1. Busca Cotizacion por ID y valida token de único uso (OTT) vía Webhook de WhatsApp (US-04). 2. Llama a cotizacion.procesarRespuesta(), donde el dominio valida exigencia del 50% de anticipo en presupuestos > $5,000 MXN (US-06). 3. Persiste con CotizacionRepository.guardar(). 4. Ejecuta UnitOfWork.commit().
OrdenServicioRepository, NotificacionService	estatusAdicionalId	1. Carga la OrdenDeServicio por ID. 2. Evalúa si el costo adicional rebasa $800 MXN (notificando al cliente) o si el acumulado supera el 30% del presupuesto (bloqueando asesor y exigiendo aprobación de Gerencia) (US-07). 3. Actualiza el estado de la orden. 4. Aplica UnitOfWork.commit().
OrdenServicioRepository, NotificacionService	cargoAplicadoDTO	1. Obtiene la OrdenDeServicio por ID. 2. Invoca orden.aplicarRecargoAlmacenaje(), donde el dominio valida que hayan transcurrido ≥ 15 días y 3 llamadas de aviso sin recolección (US-13). 3. Genera el Value Object con el cálculo monetario ($100 MXN/día). 4. Actualiza la orden y ejecuta UnitOfWork.commit().

3.1 Repositorios

Repositorio del Dominio	Método / Firma del Contrato	Intención del Negocio	Parámetros y Retorno
ClienteRepository	obtenerPorId	Recupera el agregado Cliente registrado para gestionar sus solicitudes de servicio.	In: id: ClienteId Out: Cliente
ClienteRepository	guardar	Persiste o actualiza el estado transaccional del cliente en el sistema.	In: cliente: Cliente Out: void
OrdenServicioRepository	obtenerPorId	Obtiene el agregado central OrdenDeServicio para modificar su ciclo de vida (recepción, reparación, cierre, resguardo).	In: id: OrdenServicioId Out: OrdenDeServicio
OrdenServicioRepository	guardar	Persiste el estado actualizado de la orden de servicio, sus líneas y registros de auditoría.	In: orden: OrdenDeServicio Out: void
CotizacionRepository	obtenerPorId	Carga el agregado Cotizacion para consultar o modificar su estado de aprobación y montos.	In: id: CotizacionId Out: Cotizacion

CotizacionRepository	guardar	Persiste las actualizaciones de la cotización, firmas digitales y registros de anticipos.	In: cotizacion: Cotizacion Out: void
TallerCapacidadRepository	obtenerEstadoActual	Ejecuta la consulta para verificar la ocupación de las 12 bahías y 8 cajones del taller.	In: void Out: TallerCapacidad

Aplicación CQRS Básico

Aquí tienes la información organizada en una sola tabla, lista para copiar y pegar en Word.

Tipo de Operación	**Tipo de Mensaje**	**Casos de Uso Representativos**	**Flujo Arquitectónico**	**Impacto en el Estado del Dominio**
Escritura	Commands (Comandos)	• RegistrarOrdenServicio Command • CerrarOrdenServicio Command • AutorizarCotizacion Command	Pasa por el Application Service, carga Agregados, valida reglas de negocio en el Dominio y persiste usando Unit of Work.	Modifica Estado: Cambia los datos en el sistema y puede emitir Domain Events.
Lectura	Queries (Consultas)	• ConsultarEstatusCliente Query • ObtenerHistorialVehiculo Query • VerDetalleOrdenQuery	Evita el Modelo de Dominio y los Agregados complejos. Consulta directo a proyecciones o vistas rápidas para retornar DTOs de lectura.	Solo Lectura: Consulta información sin modificar el estado del dominio.

Cumplimiento de los Principios SOLID

Aquí tienes la información organizada en una sola tabla, lista para copiar y pegar en Word.

Principio SOLID	**Definición del Principio**	**Aplicación Concreta en la Arquitectura de AutoFix Pro**	**Beneficio para el Código**
S - Single Responsibility (Responsabilidad Única)	Una clase debe tener una sola razón para cambiar.	Cada Application Service orquesta únicamente su módulo de casos de uso (ej. OrdenServicioAppService). El cálculo de recargos por almacenaje vive exclusivamente en su Value Object.	Evita clases monolíticas «Dios» y facilita el mantenimiento.
O - Open/Closed (Abierto / Cerrado)	Abierto a extensión, cerrado a modificación.	Si cambia una regla comercial (ej. el monto de recargo por almacenaje o umbrales de anticipo), se crea un nuevo Domain Service o Value Object sin modificar la capa de servicios ni repositorios.	Previene regresiones o fallas en funcionalidades existentes.
L - Liskov Substitution (Sustitución de Liskov)	Los subtipos deben ser sustituibles por sus clases base.	Los servicios dependen de la abstracción OrdenServicioRepository. Podemos cambiar la implementación de base de datos (SQL, NoSQL, en memoria) sin romper el servicio.	Permite intercambiar tecnologías de almacenamiento sin tocar la lógica.
I - Interface Segregation (Segregación de Interfaces)	Muchas interfaces específicas son mejores que una general.	Las interfaces de los repositorios son pequeñas y están orientadas estrictamente a las necesidades de cada Agregado (Cliente, OrdenDeServicio, Cotizacion).	Evita obligar a las clases a implementar métodos que no utilizan.

D - Dependency Inversion (Inversión de Dependencias)	Depender de abstracciones, no de concreciones.	La capa de servicios y el dominio dependen de interfaces abstractas. La infraestructura concreta (base de datos o pasarelas) implementa dichas interfaces.	Desacoplamiento del framework y de la base de datos.